

Moving this site off Runable and onto Vercel

Everything here is written to be typed on your own laptop, in order, top to bottom. Nothing is assumed. Where a command does something non-obvious, the line underneath says what it did.

Time: about an hour the first time. Cost: nothing, everything below has a free tier that this site will not come close to exhausting.

0. What you are actually moving

Your site is not a folder of HTML files. It is two things that happen to live in one repo:

Piece	What it is	Where it goes on Vercel
The site	React, compiled by Vite into plain files	Vercel's CDN, as static files
The API	A Hono server: <code>/studio</code> , the guestbook, view counts, the GitHub and Letterboxd feeds, photo uploads	A Vercel Function, one file, runs on demand
The database	Turso (SQLite over HTTP). Holds guestbook entries, view counts, and every edit you make in <code>/studio</code>	Your own Turso account
The photo bucket	Tigris (S3-compatible). Holds photos you upload in <code>/studio</code>	Your own Tigris account

On Runable, all four run inside one always-on Bun process, and the database and bucket belong to Runable. That is what your credits pay for.

On Vercel, the first two are free forever at your traffic. The last two need to become **yours**, because you cannot keep using Runable's. That is the only real work in this document.

What this means in practice: the guestbook entries and the `/studio` edits currently live in Runable's database and will not follow you. Your uploaded photos live in Runable's bucket and will not follow you either. After the move you will log into your own `/studio` and re-upload the photo and re-enter any text you changed. It takes five minutes. Step 8 covers it.

1. Get the tools onto your laptop

Open Terminal (macOS: Cmd+Space, type "Terminal", Enter).

Check what you already have. Run each line, one at a time:

```
git --version
node --version
bun --version
```

Any that answers with a version number is already installed. For any that says

`command not found`, install it:

- **git** — macOS: `xcode-select --install`. Windows: download from git-scm.com.
- **node** — download the LTS build from nodejs.org.
- **bun** — `curl -fsSL https://bun.sh/install | bash`, then **close and reopen Terminal** so the new command is found.

You need one more, a CLI for the database. Install it now:

```
curl -sSfL https://tur.so/install.sh | bash
```

Close and reopen Terminal again, then check it: `turso --version`.

2. Get the code

The code is already on GitHub at github.com/aryqn13/portfolio, pushed from Runnable. You are going to copy it down to your laptop.

```
cd ~
mkdir -p code
cd code
```

`cd` = change directory. `~` is your home folder. You just made a `code` folder and stepped into it. Everything below happens inside it.

```
git clone https://github.com/aryqn13/portfolio.git
cd portfolio
```

git clone downloads the whole repo including its history. You now have a folder `~/code/portfolio` that is a full copy, already connected to GitHub.

Check it worked:

```
git status
```

Should say On branch main and nothing to commit, working tree clean. That sentence means: you have no unsaved changes. Get comfortable with this command, it is the one you will run most.

Install the dependencies:

```
bun install
```

Reads package.json, downloads every library the project needs into node_modules/. Takes a minute. node_modules is never committed to git, which is why you have to do this after cloning.

3. Make your own database (Turso)

```
turso auth signup
```

Opens your browser. Sign in with GitHub. Free tier: 5 GB and 500 million row reads a month. This site will use a rounding error of that.

Create the database:

```
turso db create aryan-portfolio
```

Now get the two values the site needs to connect to it:

```
turso db show aryan-portfolio --url
turso db tokens create aryan-portfolio
```

The first prints a URL starting `libsql://`. The second prints a long token string. **Copy both into a note somewhere**, you need them twice: once in the next step and once in Vercel.

Create your local settings file:

```
cp .env.template .env
```

`.env` is where secrets live. It is listed in `.gitignore`, so it never gets committed and never reaches GitHub. That is deliberate and you should keep it that way. `.env.template` is the blank version that is committed, so you know which names to fill in.

Open `.env` in a text editor. Fill in the two database lines, and **add the four lines at the bottom** — they are not in the template because Runnable supplies them another way:

```
DATABASE_URL=libsql://...      # from `turso db show`
DATABASE_AUTH_TOKEN=...        # from `turso db tokens create`

# add these four yourself
ADMIN_PASSWORD=pick-a-good-one # this is your /studio login
ADMIN_SESSION_SECRET=...       # any long random string, see below
WEBSITE_URL=http://localhost:3000
NODE_ENV=development
```

Leave every other line in the file blank. `BETTER_AUTH_SECRET`, `AI_GATEWAY_*` and `AUTUMN_SECRET_KEY` belong to Runnable platform features this site does not use.

For that last one, generate something random:

```
openssl rand -hex 32
```

Now create the tables inside your empty database:

```
bun run db:push
```

Reads the table definitions in `packages/web/src/api/database/schema.ts` and creates them for real in Turso. Run this again any time those definitions change.

4. Make your own photo bucket (Tigris)

Photos uploaded in `/studio` cannot live in the repo, because you upload them after the site is built. They need object storage.

1. Go to storage.new (that is Tigris) and sign up. Free tier is 5 GB, no card.
2. Create a bucket. Name it `aryan-portfolio`. Leave it **private**, the site reads from it server-side.
3. Create an access key. You get an **Access Key ID** and a **Secret Access Key**. The secret is shown once, copy it now.

Add these four lines to `.env` as well:

```
S3_ENDPOINT=https://t3.storage.dev
S3_BUCKET=aryan-portfolio
S3_ACCESS_KEY_ID=...
S3_SECRET_ACCESS_KEY=...
```

*`https://t3.storage.dev` is Tigris's S3 endpoint and is the same one this site already uses on Runnable, so it is the one value you can copy blind.
If you would rather use Cloudflare R2, it works identically — it is the same S3 API. Only `S3_ENDPOINT` changes.*

5. Run it on your own machine

Before deploying anything, prove it works locally.

```
bun run build
bun run --cwd packages/web start
```

*The first compiles the React app into `packages/web/dist` . The second starts the server. Open **`http://localhost:3000`**.*

Test the parts that need the database and bucket:

- Go to `/studio` , log in with the `ADMIN_PASSWORD` you set.
- Upload your photo. If it appears, Tigris works.
- Sign the guestbook on `/elsewhere` . If it saves, Turso works.

Stop the server with `ctrl+C` .

If both worked, the hard part is done. Vercel is now just a place to put it.

6. Take the Runnable badge off

The "Made with Runnable" badge is in the code because the site was built there. It is yours to remove.

Open `packages/web/src/web/app.tsx` and delete these three lines:

```
import { AgentFeedback, RunnableBadge } from "@runablehq/website-runtime";
{import.meta.env.DEV && <AgentFeedback />}
{<RunnableBadge />}
```

Then save your change to git:

```
git add -A
git commit -m "Remove Runnable badge"
git push
```

`git add -A` stages every change. `git commit -m "..."` saves them as one labelled point in history. `git push` sends that to GitHub. Those three commands, in that order, are 90% of git.

7. Deploy to Vercel

1. Go to vercel.com and sign up **with GitHub**.
2. Click **Add New** → **Project**.
3. Find `aryqn13/portfolio` in the list and click **Import**.
4. **Do not touch the build settings**. The `vercel.json` in the repo already tells Vercel everything: install with bun, build the web package, serve `packages/web/dist`, and send `/api/*` to the function in `api/index.ts`.
5. Expand **Environment Variables** and add all six from your `.env`:

Name	Value
<code>DATABASE_URL</code>	your <code>libsql://...</code>
<code>DATABASE_AUTH_TOKEN</code>	your Turso token
<code>S3_ENDPOINT</code>	<code>https://t3.storage.dev</code>
<code>S3_BUCKET</code>	<code>aryan-portfolio</code>
<code>S3_ACCESS_KEY_ID</code>	your Tigris key id
<code>S3_SECRET_ACCESS_KEY</code>	your Tigris secret
<code>ADMIN_PASSWORD</code>	your <code>/studio</code> password
<code>ADMIN_SESSION_SECRET</code>	your random string

*Vercel never reads your `.env` file — it is not in the repo, on purpose.
This table is how the same values get to the server.*

1. Click **Deploy**. Two or three minutes.

You get a URL like `portfolio-xyz.vercel.app`. Open it.

8. Put your content back

The new database is empty and the new bucket has no photos.

1. Go to `your-url.vercel.app/studio`.
2. Log in with `ADMIN_PASSWORD`.
3. Upload your photo under **Photo deck**.

4. Re-enter anything else you had edited.

Content you never edited comes from the code itself and is already correct.

9. Your own domain

Buy a domain (Namecheap, Cloudflare, Porkbun — about ₹900/year for a `.com`).

In Vercel: **Project** → **Settings** → **Domains** → **Add**. Type the domain. Vercel shows you two DNS records to create at your registrar. Create them, wait, done — HTTPS is automatic and free.

10. How you work from now on

This is the loop, forever:

```
cd ~/code/portfolio
# ...make a change in your editor...
bun run dev          # check it at localhost:5173 while you work
git add -A
git commit -m "what you changed"
git push
```

The push triggers Vercel automatically. About a minute later it is live. There is no deploy button.

Two things worth knowing:

- **Text and photo changes do not need any of this.** Use `/studio` — it writes to the database, and it is live immediately. Only code changes need a push.
 - **Every push to a branch that is not `main` gets its own preview URL.** So you can try something risky without touching the live site:
`git checkout -b trying-something`, push, and Vercel gives you a separate URL.
-

11. When something breaks

Build fails on Vercel. Open the deployment, read the log, find the first red line. Nine times in ten it is a missing environment variable.

Site loads but the guestbook and `/studio` return errors. The frontend is fine, the function is not. Vercel → your project → **Logs**, and watch while you reload the page. Usually `DATABASE_URL` or `DATABASE_AUTH_TOKEN` is wrong or missing.

`/studio` will not accept your password. `ADMIN_PASSWORD` in Vercel does not match what you are typing. Environment variable changes need a **redeploy** to take effect — Vercel does not apply them to the running deployment.

Photos upload but do not appear. Bucket name or keys are wrong, or the key lacks write permission. Check the same values in Tigris.

The API 404s on every route. Something is wrong with `vercel.json`'s rewrites, or Vercel could not build `api/index.ts`. Check that the Root Directory in Vercel's project settings is empty (the repo root), not `packages/web`.

12. Quick reference

Command	What it does
<code>git status</code>	What have I changed?
<code>git add -A</code>	Stage all changes
<code>git commit -m "..."</code>	Save them as a labelled point
<code>git push</code>	Send to GitHub, which deploys
<code>git pull</code>	Get changes made elsewhere
<code>git log --oneline</code>	History, newest first
<code>bun install</code>	Install dependencies
<code>bun run dev</code>	Dev server, localhost:5173
<code>bun run build</code>	Compile for production
<code>bun run db:push</code>	Apply schema changes to Turso
<code>turso db shell aryan-portfolio</code>	Poke at the database in SQL